

Maximum Sum of K-Sized Subarray

Given an array of N integers and a positive integer K , design an algorithm to find **maximum** sum of a **K-sized** subarray.

Example

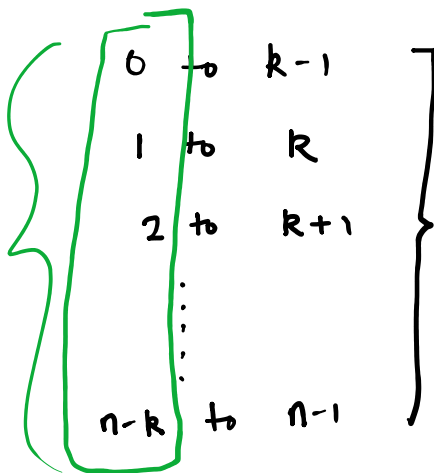
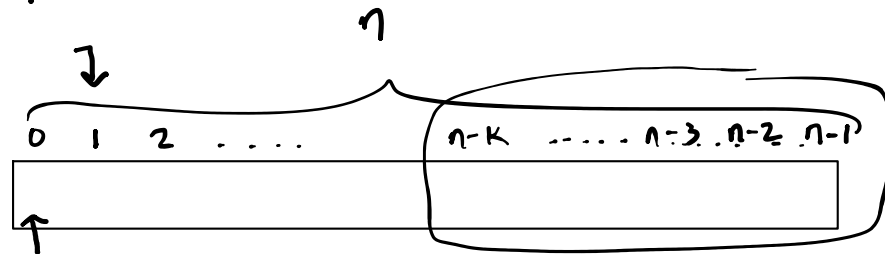
Input : $N = 7, K = 4$

0	1	2	3	4	5	6
1	4	2	7	6	3	5

Output : 21

$$\begin{aligned}
 [1 \quad 4 \quad 2 \quad 7] &= 14 \\
 [4 \quad 2 \quad 7 \quad 6] &= 19 \\
 [2 \quad 7 \quad 6 \quad 3] &= 18 \\
 [7 \quad 6 \quad 3 \quad 5] &= 21
 \end{aligned}$$

max = 21



$$(n-k) - 0 + 1 = \underline{\underline{n-k+1}}$$

subarrays of size k

$\Downarrow$
 $(n-k+1) \cdot k$ time spent to find sum of any k -sized subarray

time:

$$nk - k^2 + k \rightarrow O(nk)$$

space: $O(1)$

1st array

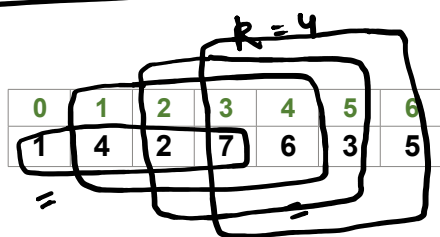
using Osum[]

to iterate over
 1 k -sized subarray

2nd appr

using $psum[]$ to iterate over k -sized subarray
time: $1 + (n-k+1) \cdot const \sim O(n)$
to pre compute $psum$
space: $O(n)$ due to $psum[]$

3rd appr : Sliding window



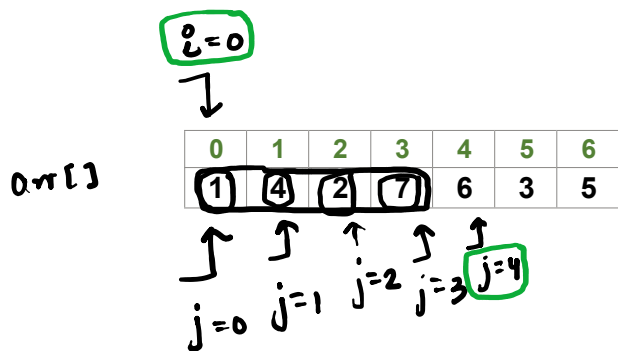
$$\Rightarrow [1, \boxed{4, 2, 7}] = 14$$

$$\Rightarrow \boxed{4, 2, 7, 6} = 19$$

$$\boxed{2, 7, 6, 3} = 18$$

$$\boxed{7, 6, 3, 5} = 21$$

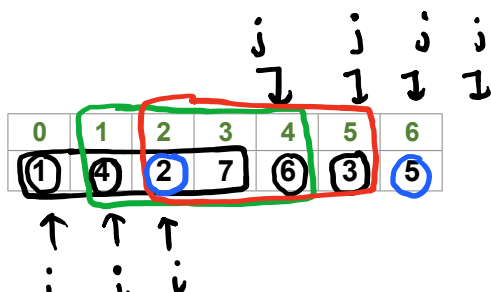
$k=4$ $msf = 14$



$wsum = 14$
while ($j < k$) {
 $wsum += arr[j];$
 $j++$;
}

$0 < 4$? ✓
 $1 < 4$? ✓
 $2 < 4$? ✓
 $3 < 4$? ✓
 $4 < 4$? ✗

$msf = 14$
 $wsum = 14$
 19
 18
 21



Slide window
 $wsum -= arr[i];$

↑ ↑ ↑
i i i

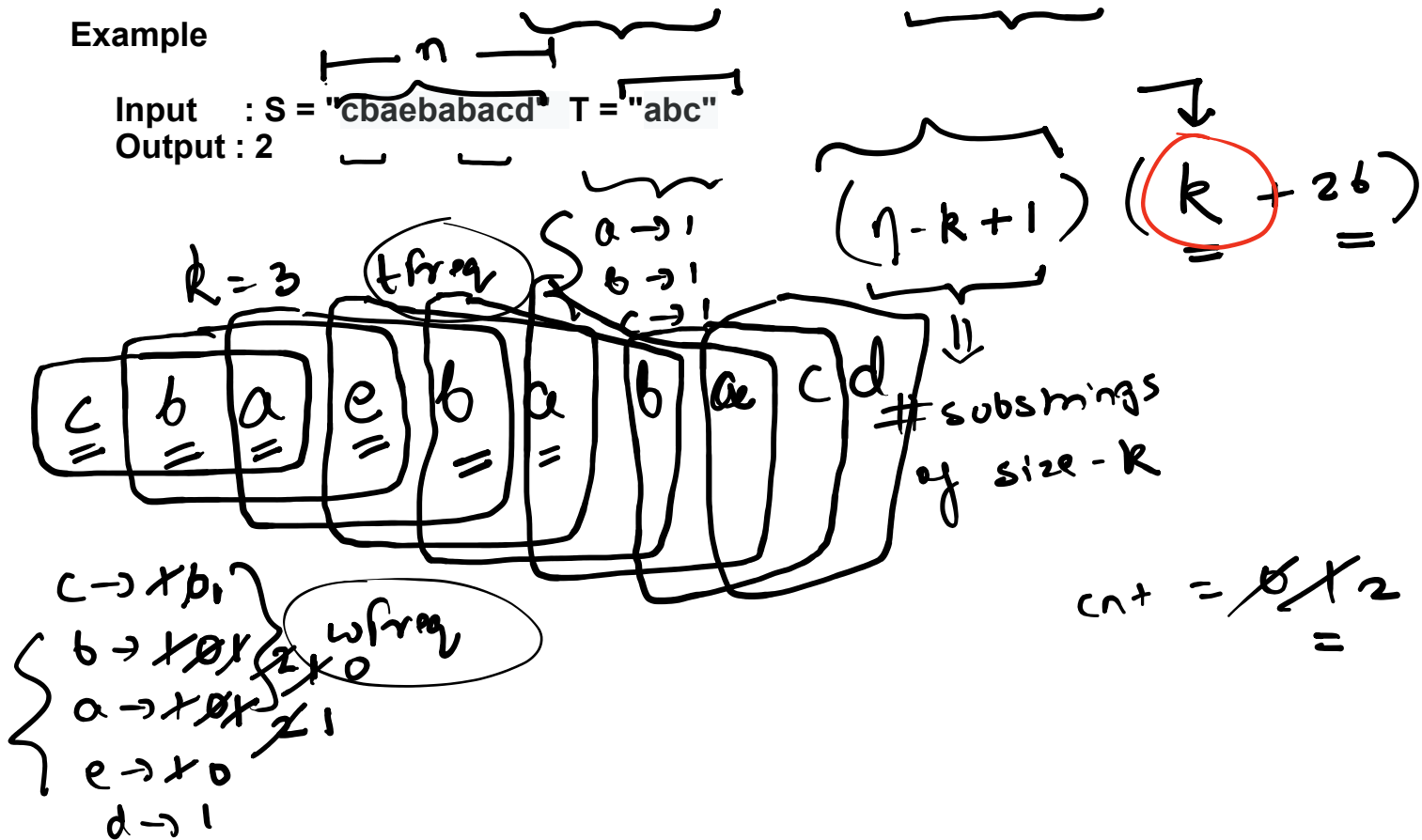
{
wsum -= a[i];
i++;
wsum += a[j];

Count Anagrams

Given two strings **S** and **T** of lengths **N** and **K** respectively, design an algorithm to count the no. of **substrings** in **S** that are **anagrams** of **T**.

Example

Input : S = "cbaebabacd" T = "abc"
Output : 2



First negative integer in a K-sized window

First negative integer in a K-sized window

Given an array of **N** integers & a positive integer **K**, design an algorithm to find the **first negative integer** for each and every **window** of size K.

note : for a window without any negative elements, output 0.

Example

Input : N = 7, K = 4

0	1	2	3	4	5	6
-2	0	-1	2	-3	1	3

Output :

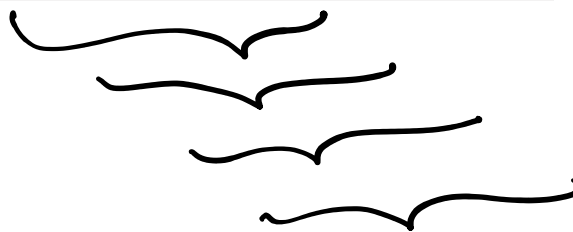
-2	-1	-1	-3
----	----	----	----

o/p:

-2, -1, -1, -3

i/p:

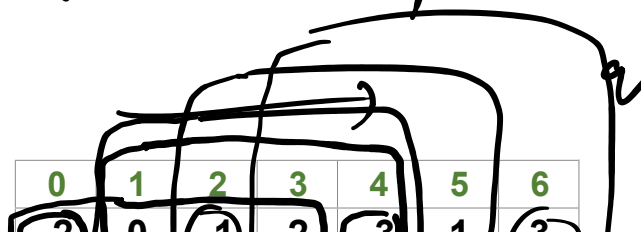
0	1	2	3	4	5	6
-2	0	-1	2	-3	1	3



time:

$$(n - k + 1) \cdot k \sim O(nk)$$

Space : $O(1)$



$$\frac{1}{-2} \cdot \frac{-1}{-3} =$$

0	1	2	3	4	5	6
-2	0	-1	2	-3	1	3

-2, -1, -1, -3

Maximum Element in a K-sized window

Given an array of **N** integers & a positive integer **K**, design an algorithm to find the **maximum integer** for each and every K-sized **window** (subarray).

Example

Input : N = 7, K = 4

0	1	2	3	4	5	6
1	4	0	1	2	3	5

Output :

4	4	3	5
---	---	---	---

Longest Subarray having sum K

Longest Subarray with Sum K

Given an array of **non-negative** integers, design an algorithm to find the **length** of the **longest** sub-array having a sum equals to K.

Example

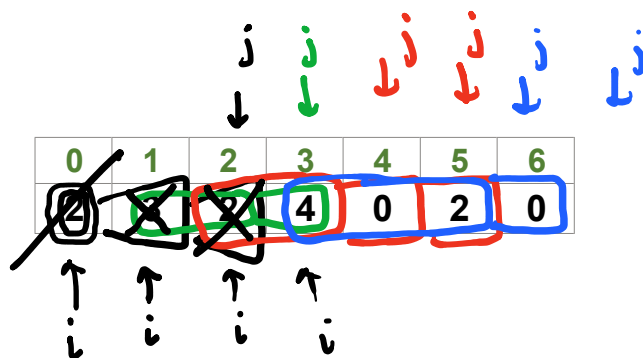
Input : N = 7, K = 6

0	1	2	3	4	5	6
2	3	2	4	0	2	0

Output : 4

msf = ~~2~~ 3 4

K = 6



wsu m = ~~2~~ ~~3~~ ~~2~~ ~~4~~ ~~0~~ ~~2~~ ~~0~~ = 6

Longest Substring without Repeating Characters

Given a strings **S** of lengths **N**, design an algorithm to compute the **length of the longest** substring without any **repeating** characters.

Example

Input : S = "abcab**cb**bb"

Output : 3





Minimum Window Substring

Given two strings **S** and **T** of lengths **M** and **N** respectively, design an algorithm to find the **length** of the **minimum window substring** of **S** that contains every character of **T** (including duplicates)

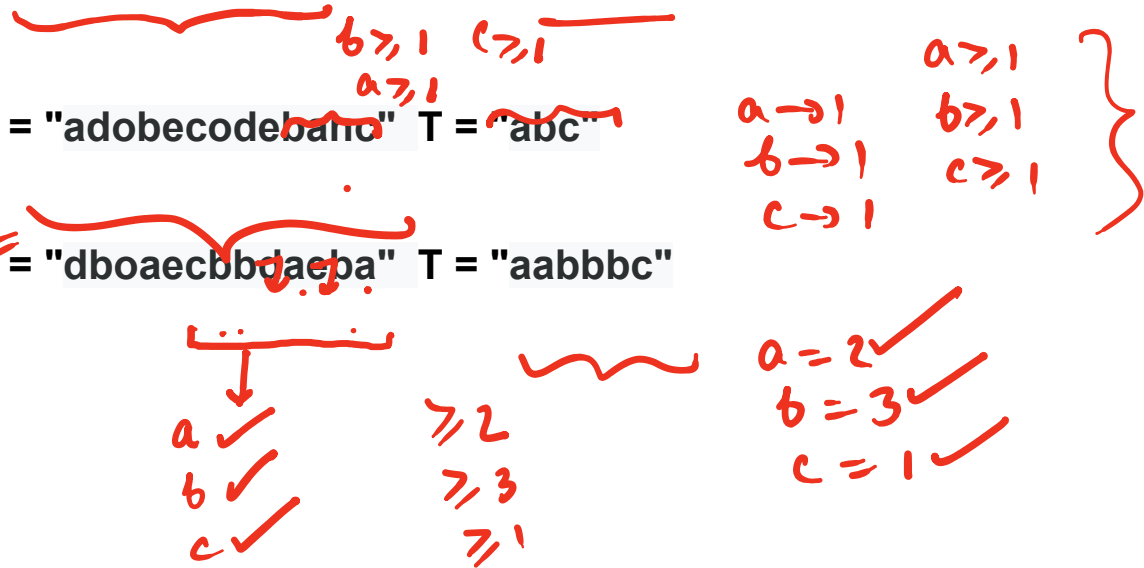
Example

Input : S = "adobecodebano" T = "abc"

Output : 4

Input : S = "dboaecbbbaeaba" T = "aabbbc"

Output : 8



[HW] Count Number of Nice Subarrays

Count Number of Nice Subarrays

Given an array of integers `nums` and an integer `k`. A continuous subarray is called **nice** if there are `k` odd numbers on it.

Return *the number of **nice** sub-arrays*.

Example 1:

Input: `nums = [1,1,2,1,1], k = 3`

Output: `2`

Explanation: The only sub-arrays with 3 odd numbers are `[1,1,2,1]` and `[1,2,1,1]`.